ARCHITECTURE AND DECISION RECORD

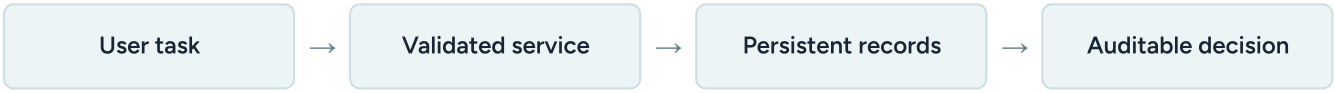
An inventory ledger and short-horizon demand dashboard

Andrea Bautista · Business systems and inventory data reviewer

Laravel | MySQL | Python | Inventory ledgers

Problem and contribution

This original design case study explores how to maintain an auditable stock ledger and provide transparent short-term forecasts whose assumptions and error are visible to the shop owner. I authored the requirements breakdown, architecture boundaries, data relationships, failure scenarios, and review rationale presented here. The design is a fictional presentation example, with no claim of a production deployment or measured client result.



Architecture decisions and tradeoffs

- Use an auditable movement ledger and idempotent transaction keys to prevent retries from double-counting inventory.
- Keep the training cutoff explicit and never use later sales when evaluating an earlier forecast.
- Compare against naive and seasonal baselines; a more complicated method is not assumed to be better.

Core records

Record	Responsibility
products	A SKU has stock movements and sales lines; unit conventions are explicit.
sales / sale_lines	Lines preserve the price at sale time instead of reading the current product price.
stock_movements	Every movement has an attributable source; correction uses compensating entries.
returns	A return references the original sale and cannot exceed eligible unreturned quantity.

The service layer owns authorization and business constraints. Interface validation improves the experience but cannot replace server checks. Event records preserve attributable decisions and their timing without logging unnecessary private content.

VERIFICATION AND REVIEW PRACTICE

Failure cases, evidence, and limitations

Proposed verification matrix

Scenario	Method	Expected behavior
Ledger consistency	Record a sale, partial return, damaged-stock adjustment, and retried receipt.	The ledger reconstructs the expected stock and ignores a duplicate source key.
Return boundary	Attempt a return larger than the original eligible quantity.	The server rejects the excess and retains a readable reason.
No future leakage	Inspect feature windows for a forecast issued at a fixed historical cutoff.	All inputs precede the cutoff and later observations are used only for evaluation.
Forecast communication	Ask users to compare forecast intervals, low-history warnings, and a baseline.	Record whether they understand uncertainty and avoid treating estimates as guaranteed sales.

Technical review

Use compensating stock movements for returns and corrections. Evaluate predictions on later time windows that were unseen at training time and compare them with a simple baseline before making a forecasting claim.

How to evaluate the design

Report stock reconciliation discrepancies, duplicate movement rejection, MAE by product and horizon, and comparison with a naive baseline. Use rolling time windows rather than random row splits. For very low sales, avoid relying on percentage errors that become unstable near zero.

The matrix specifies intended tests, not completed test results. A reviewer should capture the environment, initial records, inputs, expected state, observed state, and any unresolved defect before interpreting the outcome. Repeated runs should use separate unique request keys where relevant and preserve failures for analysis.

Scope limitations

No payment processing, accounting certification, guaranteed revenue improvement, automated purchasing, or forecast claim based on unmeasured real-shop results.

Review ledger correctness before forecast sophistication, then confirm that the proposed time-series evaluation prevents future-data leakage.

> **Authorship and reuse**
>
> Prepared by Andrea Bautista as an original presentation work sample. All narrative, schemas, and test scenarios are original illustrative material. There are no external client documents, employment certificates, real personal records, or fabricated citations in this case study.